

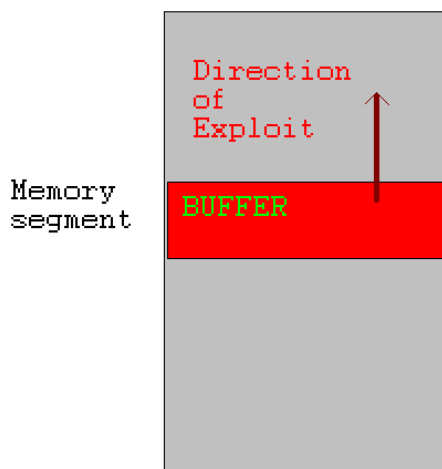
第七讲 指针安全

● 指针安全是通过修改指针值来利用程序漏洞的方法的统称

→ 可以通过覆盖函数指针将程序的控制权转移到攻击者提供的 shellcode

→ 对象指针也可以被修改，从而执行任意代码

● 缓冲区溢出覆写指针条件：



→ 缓冲区与目标指针必须分配在同一个段内

→ 缓冲区必须位于比目标指针更低的内存地址处

→ 该缓冲区必须是界限不充分的，因此容易被缓冲区溢出利用

● UNIX 可执行文件包含 data 段和 BSS 段

● data 段包含了所有已初始化的全局变量和常数

● BSS (Block Started by Symbols) 段包含了所有未初始化的全局变量

● 已初始化的全局变量和未初始化变量分开是为了让汇编器不将未初始化的变量内容写入目标文件

● 程序存在漏洞，可被缓冲区溢出利用

● 缓冲区和函数指针都未初始化，因此存在于 BSS 段

一、函数指针攻击

```
1. void good_function(const char *str) {...}
2. void main(int argc, char **argv) {
3.   static char buff[BUFSIZE];
4.   static void (*funcPtr)(const char *str);
5.   funcPtr = &good_function;
6.   strncpy(buff, argv[1], strlen(argv[1]));
7.   (void)(*funcPtr)(argv[2]);
8. }
```

静态字符数组buff

funcPtr都是未初始化的，
并且存储于BSS段

```
1. void good_function(const char *str) {...}
2. void main(int argc, char **argv) {
3.   static char buff[BUFSIZE];
4.   static void (*funcPtr)(const char *str);
5.   funcPtr = &good_function;
6.   strncpy(buff, argv[1], strlen(argv[1]));
7.   (void)(*funcPtr)(argv[2]);
8. }
```

当argv[1]的
长度大于
BUFSIZE的
时候，就会
发生缓冲区
溢出

```
1. void good_function(const char *str) {...}
2. void main(int argc, char **argv) {
3.   static char buff[BUFSIZE];
4.   static void (*funcPtr)(const char *str);
5.   funcPtr = &good_function;
6.   strncpy(buff, argv[1], strlen(argv[1]));
7.   (void)(*funcPtr)(argv[2]);
8. }
```

当程序执行到funcPtr标识的函数的时候，
shellcode将会取代good_function()得以
执行

二、对象指针攻击

1、对象指针

```
void foo(void * arg, size_t len)    {
    char buff[100];
    long val = ...;
    long *ptr = ...;
    memcpy(buff, arg, len);
    *ptr = val;
    ...
    return;
}
```

缓冲区容易被漏洞利用

在溢出缓冲区后，攻击者可以覆写ptr和val

会发生任意内存写

● “任意内存写”可以改变程序控制流

● 如果指针长度等于重要的数据结构长度，任意内存写会更容易

→ Intel 32 Architectures:

➤ $\text{sizeof}(\text{void}^*) = \text{sizeof}(\text{int}) = \text{sizeof}(\text{long}) = 4\text{B}$

2、修改指令指针

● Instruction Counter (IC) (a.k.a Program Counter (PC))存储了将要执行的下一条指令地址

→ Intel: EIP 寄存器

● IC 不能被直接访问

● IC 在顺序执行代码时递增，也可以由控制转移指令间接修改

→ jmp

→ Conditional jumps

→ call

→ ret

```

int _tmain(int argc, _TCHAR* argv[]) {
    if (argc !=1){
        printf("Usage: prog_name\n");
        exit(-1);
    }
    static void (*funcPtr) (const char *str);
    funcPtr = &good_function;
    (void) (*funcPtr) ("hi ");
    good_function("there!\n");
    return 0;
}

```

使用指针调用good function

直接调用good function

```

void (*funcPtr) ("hi ");
00424178 mov esi, esp
0042417A push offset string "hi" (46802Ch)
0042417F call dword ptr [funcPtr (478400h)]
00424185 add esp, 4
00424188 cmp esi, esp

```

第一个调用good function
机器码
ff 15 00 84 47 00
最后4个字节包含了被调用函数的地址

```

good_function("there!\n");
0042418F push offset string "there!\n" (46802Ch)
00424194 call good_function (422479h)
00424199 add esp, 4

```

第二个调用good function
机器码
e8 e0 e2 ff ff
最后4字节被调用函数的相对偏移量

● 静态调用对于函数地址使用立即数

- 指令中地址被编码
- 计算地址，然后放入 IC
- 不改变执行指令，IC 不会改变

● 通过函数指针的调用是间接引用

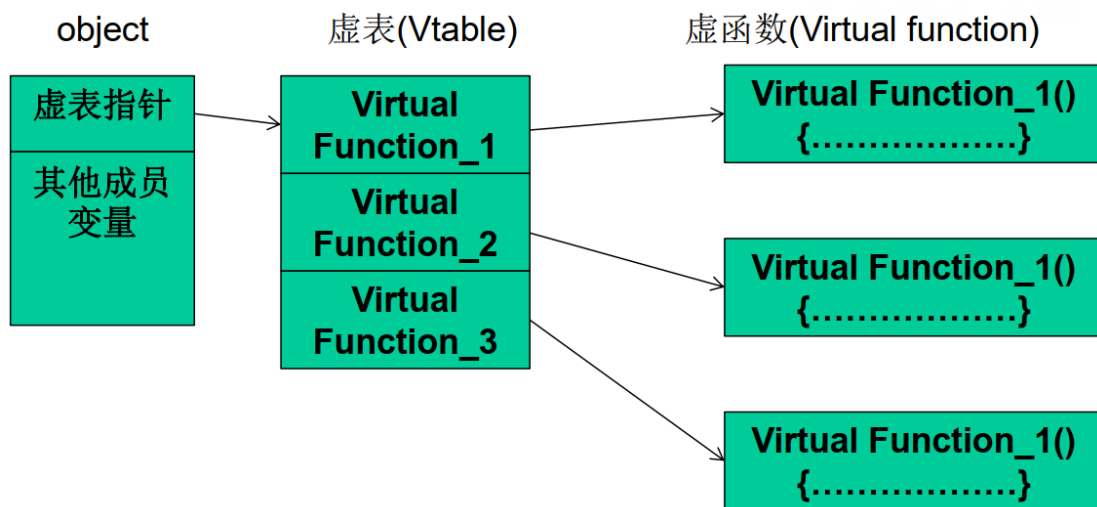
- IC 的下一个值，存储在内存中，其可以被改变

● 控制 IC 使得攻击者可以选择要执行的代码

- 攻击者能够任意写的话，很容易
- 间接的函数引用与无法在编译期间决定的函数调用可以被利用，从而使程序的控制权转移到任意代码

三、虚函数攻击

- 1) C++类的成员函数在声明时，若使用关键字 virtual 进行修饰，则被称为虚函数。
- 2) 一个类中可能有很多个虚函数。
- 3) 虚函数的入口地址被统一保存在虚表(Vtable)中。
- 4) 对象在使用虚函数时，先通过虚表指针找到虚表，然后从虚表中取出最终的函数入口地址进行调用。
- 5) 虚表指针保存在对象的内存空间中，紧接着虚表指针的是其他成员变量。
- 6) 虚函数只有通过对象指针的引用才能显示出其动态调用的特性。



实例见 PPT。



如果内存中存在多个对象且能够溢出到下一个对象空间中去，“连续性覆盖”还是有攻击的机会的，比如左图这种情况。

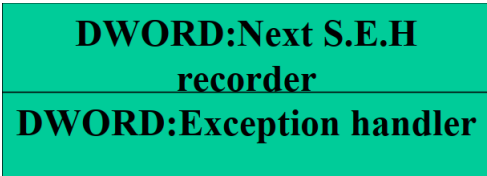
四、SEH 攻击

操作系统或程序在运行时，难免会遇到各种各样的错误，如除0、非法内存访问、文件打开错误、内存不足、磁盘读写错误、外设操作失败等。

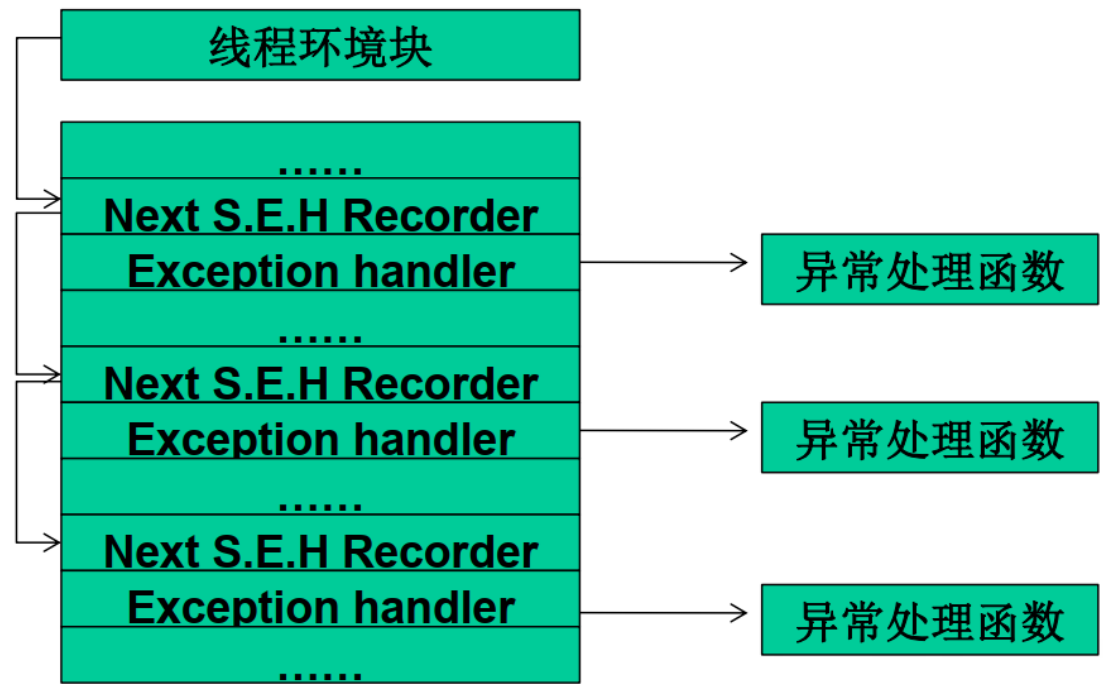
为了保证系统在遇到错误时不至于崩溃，仍能够见状稳定地继续运行下去，Windows 会对运行在其中的程序提供一次补救的机会来处理错误，这种机制就是异常处理机制。

S.E.H 即异常处理结构体(Structure Exception Handler)它是 Windows 异常处理机制所采用的重要数据结构。

每个 S.E.H 包含两个 DWORD 指针：S.E.H 链表指针和异常处理函数句柄，共 8 个字节，如图所示：



S.E.H 链表如下图所示：



1) S.E.H 结构体存放在系统栈中。

2) 当线程初始化时，会自动向栈中安装一个 S.E.H，作为线程默认的异常处理。

3) 如果程序源代码中使用了 `__try{}__except{}` 或者 `Assert` 宏等异常处理机制，编译器将最终通过向当前函数栈帧中安装一个 S.E.H 来实现异常处理。

4) 栈中一般会同时存在多个 S.E.H。

5) 栈中的多个 S.E.H 通过链表指针在栈内由栈顶向栈底串成单向链表，位于链表最顶端的 S.E.H 通过 T.E.B(线程环境块)0 字节偏移处的指针标识。

6) 当异常发生时，操作系统会中断程序，并首先从 T.E.B 的 0 字节偏移处取出距离栈顶最近的 S.E.H，使用异常处理函数句柄所指向的代码来处理异常。

7) 当离“事故现场”最近的异常处理函数运行失败时，将顺着 S.E.H 链表依次尝试其他的异常处理函数。

8) 如果程序安装的所有异常处理函数都不能处理，系统将采用默认的异常处理函数。通常，这个函数会弹出一个对话框，然后强制关闭程序。

利用 S.E.H 来进行攻击的思路：

首先，S.E.H 存放在栈内，我们学过的栈溢出漏洞可能会派上用处。

其次，我们可以精心制造出溢出数据来把 S.E.H 的异常处理的函数地址替换为 shellcode 的起始地址。

再次，溢出后错误的栈帧或堆块数据往往会触发异常，或者我们能查到哪些能触发异常的片段。

最后，当 Windows 开始处理溢出后的异常时，会调用 shellcode，它会把 shellcode 当作异常函数来处理异常。

实验见 PPT。

在 S.E.H 方面，微软也做了一些改进，比如 SafeSEH 和 SEHOP。

在 Windows XP SP2 及后续版本的操作系统中，微软引入了著名的 S.E.H 校验机制 SafeSEH。

SafeSEH 的原理很简单，在程序调用异常处理函数之前，对要调用的异常处理函数进行一系列的有效性校验，当发现异常处理函数不可靠时将终止异常处理函数的调用。

SafeSEH 需要操作系统和编译器的双重支持，二者缺一都会降低 SafeSEH 的保护能力。

SafeSEH 的保护措施：

- 1) 检查异常处理链是否位于当前程序的栈中。如果不在当前栈中，程序将终止异常处理函数的调用。
- 2) 检查异常处理函数指针是否指向当前程序的栈中，如果指向当前栈中，程序将终止异常处理函数的调用。
- 3) 在前面两项检查都通过后，程序调用一个全新的函数 `RtlIsValidHandler()`，来对异常处理函数的有效性进行验证，稍后我们会详细介绍 `RtlIsValidHandler()` 函数。

首先该函数判断异常处理函数地址是不是在加载模块的内存空间，如果属于加载模块的内存空间，校验函数将依次进行如下校验。

1) 判断程序是否设置了 IMAGE_DLLCHARACTERISTICS_NO_SEH 标识。如果设置了这个标识，这个程序内的异常会被忽略。所以当这个标识被设置时，直接返回失败。

2) 检验程序是否包含安全 S.E.H 表。如果程序包含安全 S.E.H 表，则将当前的异常处理函数地址与该表进行匹配，匹配成功则返回校验成功，匹配失败则返回校验失败。

3) 判断程序是否设置 Ilonly 标识。如果设置了这个标识，说明该程序只包含 .NET 编译人中间语言，函数直接返回校验失败。

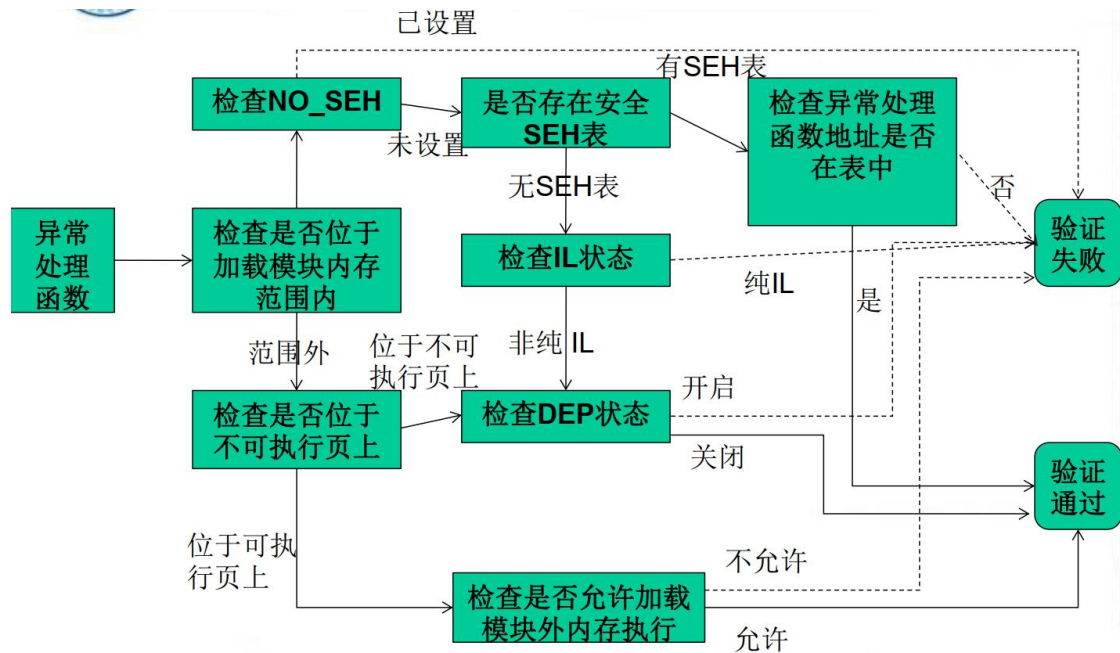
4) 判断异常处理函数地址是否位于不可执行页上。当异常处理函数地址位于不可执行页上时，校验函数将检测 DEP 是否开启，如果系统未开启 DEP 则返回校验成功，否则程序抛出访问违例的异常。

如果异常处理函数的地址没有包含在加载模块的内存空间，校验函数将直接进行 DEP 相关检测，函数依次进行如下校验。

1) 判断异常处理函数地址是否位于不可执行页上。但异常处理函数位于不可执行页上时，校验函数将检测 DEP 是否开启，如果系统未开启 DEP 则返回校验成功，否则抛出访问违例异常。

2) 判断系统是否允许跳转到加载模块的内存空间外执行，如果允许则返回校验成功，否则返回校验失败。

那我们来看看下面的流程图：

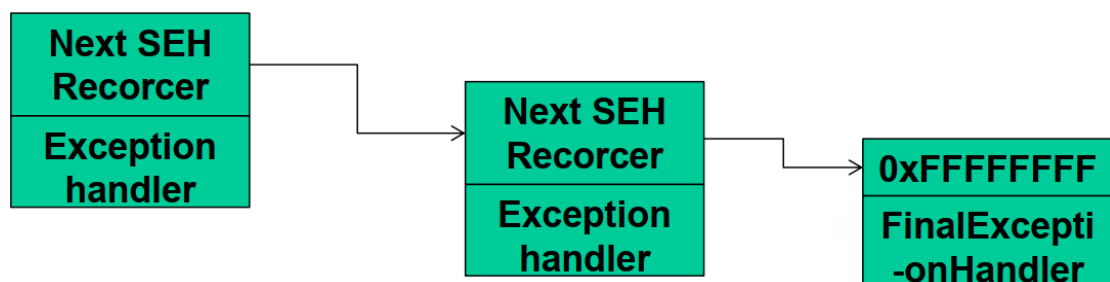


SEHOP 是一种比 SafeSEH 更为严厉的保护机制。

目前 Vista SP1 和 Windows 7 支持 SEHOP。

我们来看看 SEHOP 是干什么的。我们知道 S.E.H 函数是以单链表的形式存放在栈中的，末端是默认异常处理。

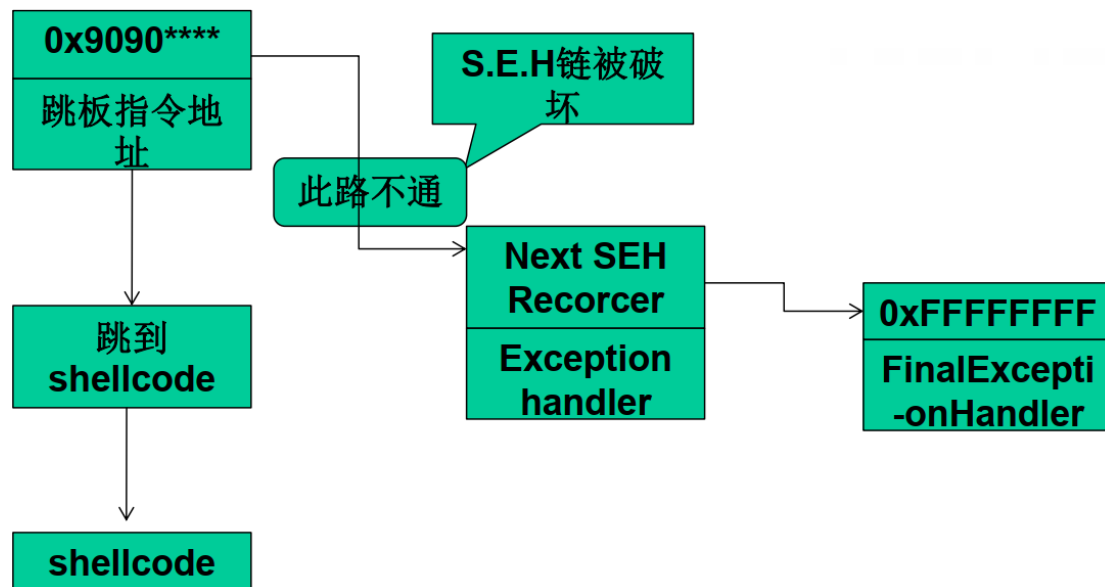
典型的 S.E.H 链：



SEHOP 的核心任务就是检查这条链的完整性，在程序转入异常处理前 SEHOP 会检查 S.E.H 链上最后一个异常处理函数是否为系统固定的终极异常处理函数。

如果是，说明这个链没有被破坏，可以去执行当前的异常处理函数；如果检测不是，则说明被破坏，程序不会去执行当前的异常处理函数。

典型的 S.E.H 溢出过程：



五、Windows 内存安全机制概述

1、从普通用户角度来看，微软在安全方面逐步做了如下几点增强。

- 1) 增加了 Windows 安全中心，提醒用户使用杀毒软件、防火墙，以及下载最新的安装补丁等。
- 2) 为 Windows 添加 PC 端的防火墙。
- 3) 未经用户允许，大多数的 Web 弹出窗口和 ActiveX 控件安装将被禁止。
- 4) IE 中增加了筛选仿冒网站功能，具有了钓鱼网站过滤器的功能。

5) 添加 UAC(User Account Control, 用户账户控制)机制, 可以防止恶意软件和间谍软件在未经许可的情况下在计算机上安装或对计算机进行更改。

6) 集成了 Windows Defender, 可以帮助阻止、控制和删除间谍软件以及其他潜在的恶意软件。

2、微软是如何提高内存保护的安全性的：

1) 使用 **GS 编译技术**, 在函数返回地址前

首先检测 Security Cookie 是否被覆盖, 从而把针对操作系统的栈溢出变得非常困难。

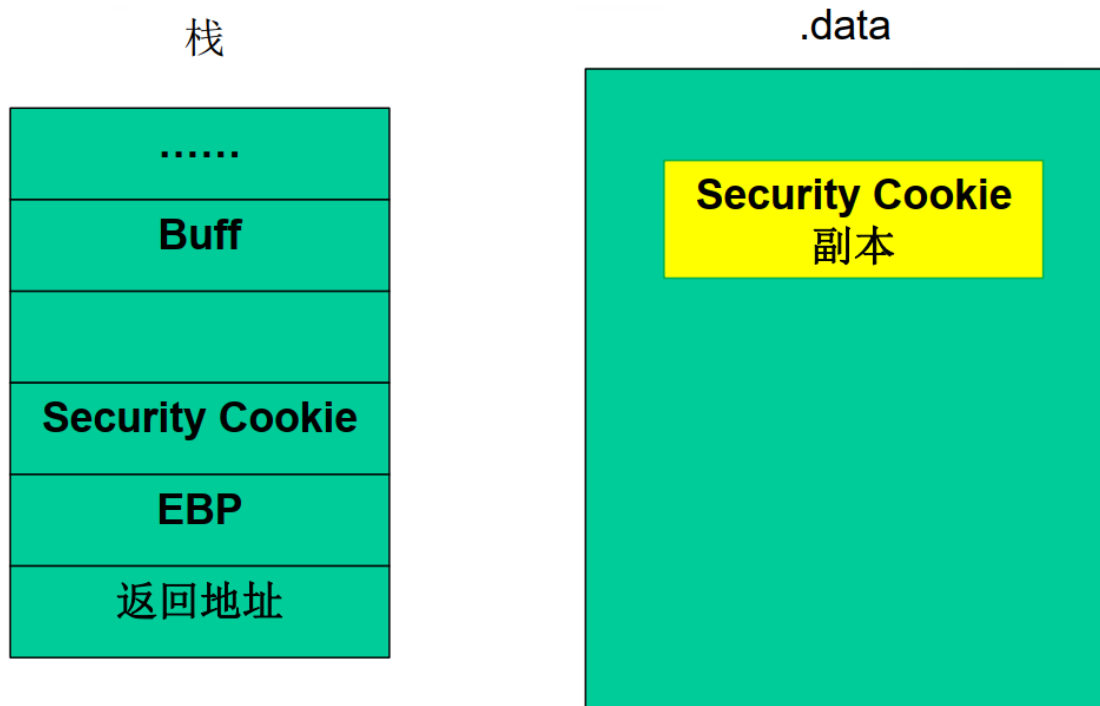
针对缓冲区溢出时覆盖返回函数地址这一特征, 微软在编译程序时使用了一个安全编译选项—GS, 在 vs2003 以及之后的版本中默认启用了这个编译选项。

GS 编译选项为每个函数调用增加了一些额外的数据和操作, 用以检测栈中的溢出。

在所有函数调用发生时, 向栈帧内压入一个额外的 DWORD, 这个随机数被称做 “canary”, 但如果使用 IDA 反汇编的话, 我们会看到 IDA 会将这个随机数标注为 “Security Cookie”。

Security Cookie 位于 EBP 之前, 系统还将在 .data 的内存区域存放一个 Security Cookie 的副本。

GS 保护机制下的内存布局：

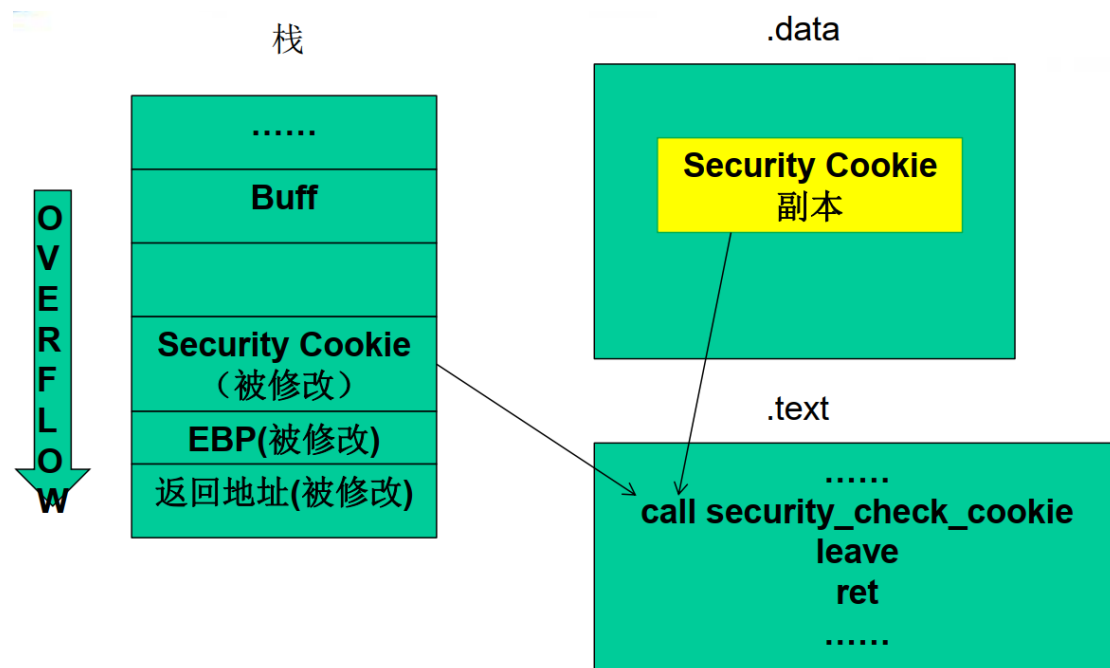


当栈中发生溢出时，首先被淹没的就是 Security Cookie，之后才是 EBP 和返回地址。在函数返回之前，系统将执行一个额外的安全验证操作，被称做 Security check。

在 Security Check 的过程中，系统将对比两个 Security Cookie 的值，如果不吻合，那么说明栈帧的 Security Cookie 已经被破坏，发生了溢出。

当检测到栈中发生溢出时，系统将进入异常处理流程，函数不会正常返回，ret 指令也不会被执行。

GS 保护机制的工作原理：



2) DEP(Data Execution Protection, 数据执行保护)将数据部分标识为不可执行, 阻止了栈、堆和数据节中攻击代码的执行。

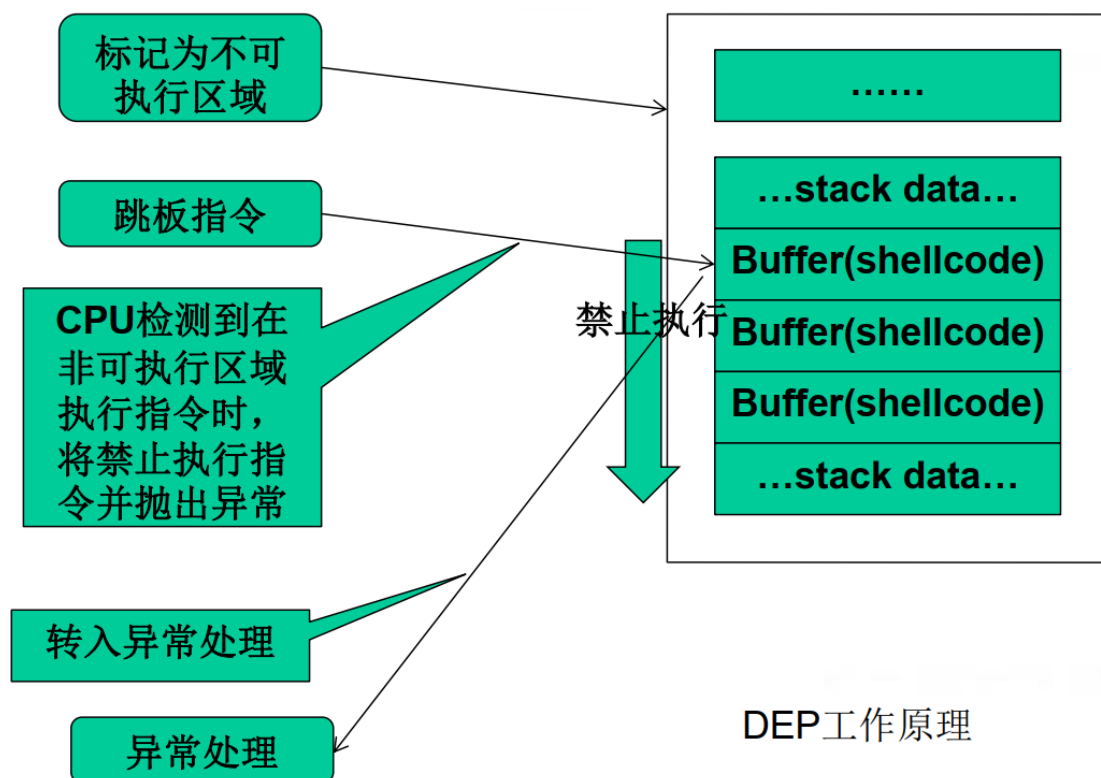
溢出攻击的根源在于现代计算机对数据和代码没有明确区分这一先天缺陷, 就目前看来重新去设计计算机体系结构基本上是不可能的, 我们只能靠向前兼容的修补来减少溢出带来的损害。

DEP 的基本原理是将数据所在内存页标识为不可执行, 当程序溢出成功转入 shellcode 时, 程序会尝试在数据页面上执行指令, 此时 CPU 就会抛出异常, 而不是去执行恶意指令。

DEP 的主要作用就是阻止数据页 (如默认的堆页、各种堆栈页及内存池页) 执行代码。

微软从 Windows XP SP2 开始提供这种技术支持。

DEP 工作原理:



DEP 根据实现机制不同可以分为：软件 DEP 和硬件 DEP。

软件 DEP 就是我们之前介绍的 SafeSEH。是 Windows 利用软件模拟实现 DEP，对操作系统提供一定的保护。

而硬件 DEP 才是真正意义的 DEP，它需要 CPU 的支持。AMD 称之为 No-Execute PageProtection(NX)，Intel 称之为 Execute Disable Bit(XD)，两者功能及工作原理在本质上是相同的。

操作系统通过设置内存页的 NX/XD 属性标记，来指明不能从该内存执行代码。

为了实现这个功能，需要在内存的页面表(Page Table)中假如一个特殊的标识位(NX/XD)来标识是否允许在该页上执行命令，0 表示允许，1 表示禁止。

3) 堆中加入了 Heap Cookie、Safe Unlinking 等一系列的安全机制，为原本就困难重重的堆溢出增加了更多的限制。

PEB(进程环境块) random: 在堆攻击中, PEB 中的函数指针是绝佳的目标。微软在 Windows XP SP2 之后不再使用固定的 PEB 基址, 而是使用具有一定随机性的 PEB 基址, PEB 随机化之后就影响了对 PEB 中函数的攻击。

Safe Unlink: 在原来卸载 free list 的堆块时, 代码可能是这样的:

```
int remove(ListNode * node){  
  
    node->blink->flink=node->flink;  
  
    node->flink->blink=node->blink;  
  
    return 0;  
  
}
```

blink 和 flink 代表前链接和后链接, 这样就有一个隐患, 当堆溢出成功时, 能够伪造出一个节点, 节点的前后 link 指向是堆外的地址, 那么卸载时就会链接到堆外, 产生漏洞。

改进后会进行验证, 以防止发生漏洞:

```
int safe_remove(ListNode * node){  
  
    if((node->blink->flink==node)&&(node->flink->blink==node)){  
  
        node->blink->flink=node->flink;  
  
        node->flink->blink=node->blink;  
  
        return 1;  
  
    }  
  
    else{  
  
        return 0;//链表被破坏
```



```
}  
  
}
```

heap cookie: 与栈中的 security cookie 类似, 微软在堆中也引入了 cookie, 用于检测堆溢出的发生。Cookie 被布置在堆首部分原堆块的 segment table 的位置, 占一个字节。

元数据加密: 微软在 Windows Vista 及后续版本的操作系统中开始使用该安全措施。块首中的一些重要数据在保存时会与一个 4 字节的随机数进行异或运算, 在使用这些数据时候需要再进行一次异或运算来还原, 这样我们就不能直接破坏这些数据了, 以达到保护堆的目的。

4) **ASLR**(Address space layout randomization, 加载地址随机)技术通过对系统关键地址的随机化, 使得经典堆栈溢出手段失效。

纵观前面介绍的所有漏洞利用方法都有一个共同的特征: 需要确定一个明确的跳转地址 ASLR 的概念在 xp 时就已经提出来了, 只不过功能很有限, 在 Vista 之后的操作系统才真正的发挥作用, 它包含了以下几个部分。

映像随机化: 映像随机化是 PE 文件映射到内存时, 对其加载的虚拟地址进行随机化处理, 这个地址是在系统启动时确定的, 系统重启后这个地址会变化。

堆栈随机化: 这项措施是在程序运行时随机的选择堆栈的基址, 与映像基址随机化不同的是堆栈的基址不是在系统启动的时候确定的, 而是在打开程序的时候确定的。也就是说同一个程序任意两次

运行时的堆栈基址都是不同的，进而各个变量在内存中的位置也是不确定的。

PEB 与 TEB(线程环境块): PEB 与 TEB 随机化在 Windows XP SP2 中就已经引入了，微软在 XP SP2 之后不再使用固定的 PEB 基址 0x7FFDF000 和 TEB 基址 0x7FFDE000，而是使用具有一定随机性的基址。但是其随机化的程度不是很好，而且即使做到了完全随机，依然还是可以通过其他方法获取到当前进程的 PEB 和 TEB。